

Modul 295 – Leistungsbeurteilung

Backend für die persönliche Spielesammlung

Aufbauend auf dem Modul-294-Projekt „Game Gallery“

Projekt: gamesbackend

Name: Remo Niederberger, Gabriell Prenaj

1. Projektidee

Die „Game Gallery“ aus Modul 294 ist eine React-Anwendung, mit der Nutzerinnen und Nutzer ihre persönliche Spielesammlung verwalten. Bisher wurden die Daten über einen simulierten json-server (db.json) gespeichert eine reine Mock-Lösung ohne echte Datenbank.

Dieses M295-Projekt ersetzt diesen Mock durch ein echtes Spring-Boot-Backend mit PostgreSQL Persistenz. Jedes Spiel (Titel, Genre, Plattform, Cover-Bild) besitzt neu nicht mehr nur eine einzelne Bewertung, sondern kann mehrere Bewertungen von verschiedenen Personen erhalten. Aus der privaten Liste wird so eine Sammlung, die zum Beispiel innerhalb einer Familie oder WG gemeinsam genutzt und bewertet werden kann.

Die REST-API bietet vollständiges CRUD für Spiele, liefert die Daten sauber über DTOs statt über die Entities, validiert Eingaben serverseitig und gibt bei Fehlern einheitliche, aussagekräftige Fehlermeldungen zurück.

2. Anforderungskatalog

Die folgenden User Stories beschreiben die Kernfunktionalität des Backends aus Sicht einer sammelnden Person.

User Story 1: Alle Spiele auflisten

Akzeptanzkriterien
GET /api/games liefert eine Liste aller Spiele inklusive ihrer Bewertungen.
Die Antwort hat den Status 200.
Ist die Sammlung leer, wird eine leere Liste zurückgegeben (kein Fehler).

User Story 2: Neues Spiel erfassen

Akzeptanzkriterien
POST /api/games mit Titel (Pflicht), Genre, Plattform, Bild-Link und optional einer Liste von Bewertungen.
Fehlt der Titel, antwortet die API mit Status 400 und einer Fehlermeldung zum Feld title.
Liegt eine Bewertung außerhalb von 1–5, antwortet die API mit Status 400 und einer Fehlermeldung zum Feld rating.
Bei Erfolg antwortet die API mit Status 201 und liefert das neu erstellte Spiel inklusive generierter id.

User Story 3: Bestehendes Spiel bearbeiten

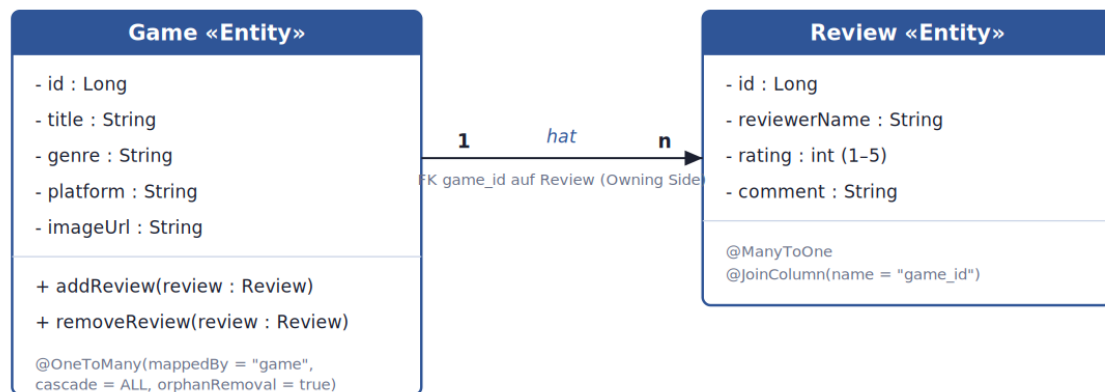
Akzeptanzkriterien
PUT /api/games/{id} ersetzt Titel, Genre, Plattform, Bild-Link und Bewertungen.
Existiert die id nicht, antwortet die API mit Status 404 und einer Fehlermeldung.
Bei Erfolg antwortet die API mit Status 200 und liefert das aktualisierte Spiel.

User Story 4: Spiel löschen

Akzeptanzkriterien
DELETE /api/games/{id} löscht das Spiel samt allen zugehörigen Bewertungen (cascade).
Bei Erfolg antwortet die API mit Status 204 (kein Inhalt).
Existiert die id nicht, antwortet die API mit Status 404.

3. Klassendiagramm

Das Datenmodell besteht aus zwei JPA-Entities mit einer 1:n-Beziehung: Ein Spiel (Game) kann mehrere Bewertungen (Review) besitzen. Die Fremdschlüssel-Spalte `game_id` liegt auf der Review-Tabelle (Owning Side). Die Beziehung ist mit `cascade` und `orphanRemoval` konfiguriert, sodass Bewertungen zusammen mit ihrem Spiel gespeichert und gelöscht werden.



Ein Spiel (Game) besitzt keine, eine oder mehrere Bewertungen (Review) — 1:n-Beziehung.

4. Ablauf je User Story (API-Aufrufe)

User Story 1 – Alle Spiele auflisten

1. Client sendet GET /api/games
2. Server antwortet mit Status 200 und folgendem Beispiel-Body:

```
[
  {
    "id": 2,
    "title": "Hollow Knight",
    "genre": "Metroidvania",
    "platform": "PC",
    "imageUrl": "",
    "reviews": [
      { "id": 1, "reviewerName": "Anna", "rating": 4, "comment": "Wunderschön, aber knackig schwer." }
    ]
  }
]
```

User Story 2 – Neues Spiel erfassen

1. Client sendet POST /api/games mit folgendem Body:

```
{
  "title": "Stardew Valley",
  "genre": "Simulation",
  "platform": "PC",
  "imageUrl": "",
  "reviews": [
    { "reviewerName": "Anna", "rating": 5, "comment": "Sehr entspannend." }
  ]
}
```

2. Der Server prüft die Eingabe mit Bean Validation und antwortet bei Erfolg mit Status 201:

```
{
  "id": 4,
  "title": "Stardew Valley",
  "genre": "Simulation",
  "platform": "PC",
  "imageUrl": "",
  "reviews": [
    { "id": 5, "reviewerName": "Anna", "rating": 5, "comment": "Sehr entspannend." }
  ]
}
```

3. Fehlerfall: Wird kein Titel mitgeschickt, antwortet der Server mit Status 400:

```
{
  "timestamp": "2026-07-10T10:15:00",
  "status": 400,
  "message": "Validierung fehlgeschlagen",
  "fieldErrors": { "title": "darf nicht leer sein." }
}
```

User Story 3 – Spiel bearbeiten

1. Client sendet PUT /api/games/2 mit vollständigem Body (analog User Story 2).
2. Der Server prüft, ob ein Spiel mit dieser id existiert.
3. Existiert es nicht: Status 404 mit Fehlermeldung „Kein Spiel mit der id 2 gefunden!“.
4. Existiert es: Status 200 mit dem aktualisierten Spiel als Body.

User Story 4 – Spiel löschen

1. Client sendet DELETE /api/games/2
2. Der Server löscht das Spiel und dank cascade/orphanRemoval automatisch auch alle zugehörigen Bewertungen.
3. Server antwortet mit Status 204 und leerem Body.

5. Testplan – manuelle Testfälle

Die folgenden fünf manuellen Testfälle prüfen die Kernfunktionen der API. Sie sind vor der Abgabe lokal (zum Beispiel mit Postman oder Insomnia) gegen die laufende Anwendung durchzuführen; die Spalte „Tatsächliches Ergebnis“ wird dabei ausgefüllt.

Nr.	Testfall	Schritte	Erwartetes Ergebnis	Tatsächliches Ergebnis
TC1	Alle Spiele auflisten	DB enthält die Seed-Spiele. GET /api/games senden.	Status 200, Liste mit mind. 3 Spielen inkl. reviews.	Status 200, Liste mit mind. 3 Spielen
TC2	Neues Spiel erstellen	Server läuft. POST /api/games mit gültigem Body (siehe Kapitel 4, US2).	Status 201, Response enthält neue id > 0.	Status 201, Response enthält neue id > 0.
TC3	Spiel mit leerem Titel	Server läuft. POST /api/games mit "title": "" senden.	Status 400, fieldErrors enthält einen Eintrag für title.	Status 400
TC4	Bestehendes Spiel aktualisieren	Spiel mit id 1 existiert. PUT /api/games/1 mit geändertem Titel.	Status 200, Response zeigt den neuen Titel.	Status 200, Response zeigt den neuen Titel
TC5	Nicht existierendes Spiel löschen	Keine id 9999 in der DB. DELETE /api/games/9999 senden.	Status 404 mit Fehlermeldung „Kein Spiel mit der id 9999 gefunden!“.	Status 404 mit Fehlermeldung „Kein Spiel mit der id 9999 gefunden!“.

Zusätzlich enthält das Projekt sieben automatisierte Unit-Tests (Repository-, Service- und Controller-Ebene), die bei jedem Build ausgeführt werden.

6. Installationsanleitung

Voraussetzungen

- IntelliJ IDEA (Ultimate oder Community)
- JDK 21
- Docker Desktop (für PostgreSQL)
- Optional: DBeaver (Datenbank ansehen), Postman oder Insomnia (API testen)

Schritte

1. Das Projekt in IntelliJ als Maven-Projekt öffnen (backend/pom.xml auswählen).
2. Ein Terminal im Projektordner öffnen und die Datenbank starten: `docker-compose up -d`
3. Die Zugangsdaten sind bereits in `src/main/resources/application.properties` hinterlegt (Benutzer `gamesUser`, Passwort `gamesPW`, Datenbank `gamesdb`) keine Anpassung nötig.
4. Die Klasse `GamesbackendApplication` öffnen und über den grünen Play-Button starten.
5. Die Anwendung läuft danach auf `http://localhost:8080`. Der `GameDataSeeder` legt beim ersten Start automatisch drei Beispiel-Spiele mit Bewertungen an.
6. Endpunkte testen, zum Beispiel GET `http://localhost:8080/api/games` (mit Postman oder Insomnia).
7. Automatisierte Tests ausführen: Rechtsklick auf `src/test/java` → Run All Tests. Der Docker-Container muss dafür laufen, da die Repository-Tests gegen die echte PostgreSQL-Datenbank arbeiten.
8. Zum Beenden: `docker-compose down` (die Daten bleiben im Docker-Volume erhalten). Mit `docker-compose down -v` werden auch die Daten gelöscht.

7. Hilfestellungen & Quellen

- Unterrichtsmaterialien des Moduls M295, Block 1 bis 7 Grundlage für Architektur, Coding-Muster und Testarten.
- Eigenes Projekt aus Modul M294 („Game Gallery“) als fachliche Grundlage und Ausgangs-Datenmodell.

KI-Einsatz

Chat gpt: Hilfe beim Codieren und Erstellen von classes (stellen wo Ki benutzt wurde sind markiert).

Claude: Hilfe beim Erstellen von der Dokumentation, (Grammatik, design) und Tests (sind im code markiert).